

FastMCP Deep Agents - Passing ChatSessionStore session_id via HTTP Headers

User prompt

give me the steps again if I wanted to use http request/headers to pass the session_id from the chatsessionstore

Assistant response

Yes. The clean flow is:

1. Get the app session from `ChatSessionStore`
2. Put that session ID into the Deep Agent runtime context
3. Use an MCP tool interceptor to copy it into an HTTP header
4. Read that header inside `@mcp.tool`

So the flow becomes:

```
FastAPI route
  |
  v
ChatSessionStore
  |
  v
session.id
  |
  v
Deep Agent runtime context
  |
  v
MCP interceptor
  |
  v
X-Chat-Session-ID header
  |
  v
FastMCP tool
```

1. Define agent runtime context

```
from dataclasses import dataclass
```

```
@dataclass
class AgentContext:
    session_id: str
```

When creating the Deep Agent:

```
agent = create_deep_agent(
    model=model,
    tools=tools,
    context_schema=AgentContext,
)
```

2. Get the session from `app.state.sessions`

Your FastAPI route already knows the browser/application session ID.

For example:

```
@app.post("/chat")
async def chat(payload: ChatRequest, request: Request):

    session = request.app.state.sessions.get(
        payload.session_id
    )

    return await request.app.state.engine.chat(
        message=payload.message,
        session=session,
    )
```

So now your engine receives the actual ChatSession:

```
session.id
```

For example:

```
abc123
```

3. Pass that ID into the Deep Agent

Inside your engine:

```
async def chat(
    self,
    message: str,
    session: ChatSession,
):

    result = await self.agent.ainvoke(
        {
            "messages": [
                {
                    "role": "user",
                    "content": message,
                }
            ],
        },
        context=AgentContext(
            session_id=session.id,
        ),
        config={
            "configurable": {
                "thread_id": session.id,
            }
        },
    )

    return result
```

At this point the agent runtime knows:

```
runtime.context.session_id
```

4. Add an MCP tool interceptor

The interceptor runs immediately before LangChain sends the tool call to the MCP server.

```
from langchain_mcp_adapters.interceptors import (
    MCPToolCallRequest,
```

```

)

async def session_interceptor(
    request: MCPToolCallRequest,
    handler,
):
    session_id = request.runtime.context.session_id

    headers = {
        **(request.headers or {}),
        "X-Chat-Session-ID": session_id,
    }

    request = request.override(
        headers=headers,
    )

    return await handler(request)

```

The important line is:

```
"X-Chat-Session-ID": session_id
```

Now your outbound MCP HTTP request contains:

```
X-Chat-Session-ID: abc123
```

5. Register that interceptor with `MultiServerMCPClient`

Where you currently create your MCP client:

```

client = MultiServerMCPClient(
    {
        "gateway": {
            "transport": "http",
            "url": "http://localhost:8000/mcp",
        }
    },
    tool_interceptors=[
        session_interceptor,
    ],
)

```

Then:

```
tools = await client.get_tools()
```

Your agent gets the same MCP tools as before.

The LLM doesn't see anything different.

6. Read the header inside your MCP tool

On your FastMCP server:

```

from fastmcp import FastMCP
from fastmcp.server.dependencies import get_http_headers

mcp = FastMCP("Example")

@mcp.tool

```

```

async def some_tool(value: str):

    headers = get_http_headers()

    session_id = headers.get(
        "x-chat-session-id"
    )

    print(f"App session: {session_id}")

    return value

```

HTTP headers are case-insensitive, but FastMCP/header libraries generally normalize them to lowercase when accessed, so:

```
headers.get("x-chat-session-id")
```

is what I'd use.

Full path

Suppose the frontend sends:

```

{
    "session_id": "abc123",
    "message": "Do something"
}

```

FastAPI gets:

```
session = app.state.sessions.get("abc123")
```

Then:

```

AgentContext(
    session_id="abc123"
)

```

The interceptor sees:

```
request.runtime.context.session_id
```

and generates:

```
X-Chat-Session-ID: abc123
```

Then inside:

```

@mcp.tool
async def some_tool(...):

```

you get:

```

headers = get_http_headers()

session_id = headers.get(
    "x-chat-session-id"
)

```

Result:

```
session_id == "abc123"
```

Visually

```
Frontend
session_id = abc123
  |
  v
FastAPI
  |
  v
app.state.sessions
  |
  |-- ChatSession(id="abc123")
        |
        v
      AgentEngine
        |
        v
    AgentContext(session_id="abc123")
        |
        v
      Deep Agent
        |
        v
    MCP interceptor
        |
        v
    X-Chat-Session-ID: abc123
        |
        v
    HTTP request
        v
    FastMCP Server
        |
        v
    @mcp.tool
        |
        v
    get_http_headers()
        |
        v
    session_id = "abc123"
```

One thing to keep in mind: the MCP tool now knows which ChatSession caused the call, but it still cannot directly access `app.state.sessions` if the MCP server is a different process. If the next thing you want is for the MCP tool to retrieve something like:

```
session.state["workspace"]
```

from that ChatSessionStore, that's the next architectural piece we'd need to connect.